Angular Signals and Reactive Primitives - Detailed Guide

WritableSignal

WritableSignal is a reactive primitive that can be updated using `set()` or `update()`.

- `set(value)`: Directly assigns a value.
- `update(fn)`: Updates based on current value.

```
const counter = signal(0);
counter.set(10);
counter.update(v => v + 1);
```

ComputedSignal

ComputedSignal derives values from other signals.

- **Lazy Execution:** Doesn't compute until accessed.
- **Memoization:** Caches value until dependency changes.

```
const a = signal(1);
const b = signal(2);
const sum = computed(() => a() + b());
```

ComputedSignal Dependency

Multiple signals can be tracked as dependencies.

```
const s1 = signal(10);
const s2 = signal(5);
const result = computed(() => s1() + s2());
```

Effects

- **Context injection** using `injectEffect()`

- **Manual cleanup** with `onCleanup()`
- **Untracked access** with `untracked()`

```
const cleanup = effect(() => {
  const id = setInterval(() => console.log("tick"), 1000);
  onCleanup(() => clearInterval(id));
});

effect(() => {
  console.log(untracked(() => signalA()));
});
```

toSignal and toObservable

Converting between observables and signals:

```
const obs$ = interval(1000);
const time = toSignal(obs$, { initialValue: 0 });

const count = signal(1);
const count$ = toObservable(count);
```

outputFrom and outputTo

```
@Output() valueChanged = outputFrom(this.value$);
outputTo(this.valueChanged, this.value$);
```

Signal Inputs

- `input()`: Reactive `@Input`
- `input.required()`: Validates input presence
- `transform`: Transform input before use

```
@Input() count = input<number>({ transform: v => parseInt(v) });
```

Model Inputs (Two-way Binding)

```
@Input({ alias: 'modelValue' }) value = model<string>();
@Output() valueChange = model<string>().change;
```

Signal-Based Queries

DOM/Template reference as signal

```
myButton = viewChild('myButtonRef');
items = viewChildren('listItems');
```

Content Queries

Access projected content using signal primitives

```
projectedContent = contentChild('slot');
multipleContent = contentChildren('projectedItem');
```

Summary Table

Feature	Description
WritableSignal	Mutable signal using `set` and `update`
ComputedSignal	Derived value with memoization
effect()	Side effects reacting to signal changes
toSignal/toObservable	Interop between signals and observables
outputFrom/outputTo	Use observables with @Output
input()	Reactive @Input binding
input.required()	Validates required inputs
model()	Two-way signal input binding
viewChild/viewChildren	Reactive template element references
contentChild/Children	Signal access to projected content

Important Interview Questions

1. What is a WritableSignal and how does it differ from a readonly signal?
2. How do you use `set()` and `update()` in signals?
3. Explain lazy execution and memoization in ComputedSignal.
4. How does Angular track dependencies in a ComputedSignal?
5. When would you use an effect and how do you clean it up?
6. What is the purpose of `untracked()` and where is it useful?
7. How does `toSignal()` differ from `toObservable()`?
8. When should you use `outputFrom()` or `outputTo()`?
9. What are the advantages of signal-based inputs using `input()`?
10. How does Angular handle two-way data binding with signals?
11. How can you access a DOM element reactively in Angular?
12. What are content queries and how do they work with signals?